

BCA Connect

Rapport complet d'état du projet

Date : 7 juin 2026

Version : Pilote bêta — Phases 1 & 2 clôturées · Phase 3 en cours

Équipe : 2 développeurs

Commit de référence : 9fece84 — branche main

Référence : Cahier des charges Readme.md §13

Plateforme pilote fonctionnelle — Lancement national en préparation

69 tests automatisés validés · 24 modules API · 6 rôles

Table des matières

- 1. Synthèse exécutive
- 2. Contexte et vision
- 3. Organisation équipe (2 développeurs)
- 4. Stack technique
- 5. Phase 1 — Conception & prototypage
- 6. Phase 2 — Développement pilote
- 7. Phase 3 — Lancement (en cours)
- 8. État par module fonctionnel
- 9. État par rôle utilisateur
- 10. Tests et qualité
- 11. Sécurité
- 12. Écarts, risques et recommandations
- 13. Parcours de démonstration
- 14. Annexes

1. Synthèse exécutive

BCA Connect est une marketplace fintech/logistique pour les chaînes d'approvisionnement en Guinée et en Afrique de l'Ouest. Le parcours principal est opérationnel de bout en bout :

Catalogue → commande → paiement/séquestre → livraison GPS → litige → crédit → SAV → achats groupés

Le projet est porté par une **équipe de 2 développeurs** travaillant en parallèle sur des postes locaux indépendants, synchronisés via Git.

Indicateur	Valeur
Phase 1 (Prototype)	100 % — Clôturée
Phase 2 (Pilote bêta)	~95 % — Clôturée
Phase 3 (Lancement)	~20 % — En cours
Backend API	24 modules routes, ~148 endpoints, 35 modèles, 23 contrôleurs
Frontend	66 pages React, 6 rôles opérationnels, PWA
Migrations SQL	7 fichiers versionnés
Tests automatisés vérifiés	69 / 69
Commit principal Phases 1+2	9fece84 (7 juin 2026)
Prêt lancement national	Non — Phase 3 requise

Phase 2 à 95 % : seul blocage externe = clés CinetPay production. Phase 3 démarrée : infra Postgres, SMS, exports, UAT.

2. Contexte et vision

BCA Connect vise à devenir une centrale d'achat virtuelle reliant ménages, TPE/PME, fournisseurs, banques, transporteurs et techniciens. Les piliers du projet sont :

- Centralisation du catalogue multicatégorie (38 catégories BCA)
- Paiements sécurisés avec séquestre (escrow) et Mobile Money Guinée
- Logistique GPS, livraisons groupées et 3 tiers de livraison
- Crédit et financement partenaire banque
- Litiges interactifs et médiation IA
- SAV, garanties et maintenance terrain (technicien)
- Achats groupés ONG/B2B

Phases de développement (Readme §13)

Phase	Objectif	État
Phase 1	Prototype : catalogue, commandes, paiements	Terminée ✓
Phase 2	Bêta pilote : logistique, litiges, crédit, SAV, achats groupés	Terminée ✓
Phase 3	Lancement : Postgres/Redis prod, SMS, UAT, CinetPay live	En cours
Phase 4	IoT, blockchain, ERP, USSD, ML géolocalisé	Non démarrée

3. Organisation équipe (2 développeurs)

Chaque développeur dispose de **sa propre machine**, de **sa base SQLite locale** et de **son fichier .env non versionné**. Le dépôt Git est le seul point de synchronisation du code.

Élément	Partagé (Git)	Local (chaque dev)
Code source & migrations	Oui	—
backend/.env.example	Oui	Copier → .env
Base SQLite backend/data/	Non	Oui
Docker / Postgres	Optionnel	Selon la machine
Journal collaboratif	DEVELOPMENT_LOG.md	Mise à jour en fin de session

Workflow recommandé

- git pull avant chaque session
- Démarrage : ./scripts/dev-backend.sh (SQLite auto) + cd frontend && npm run dev
- Coordination via DEVELOPMENT_LOG.md et EQUIPE_DEV.md
- Commits petits ; jamais de .env ni de database.sqlite

Ports standard équipe

Service	Port
Frontend Vite	3002
Backend API	5001
Postgres Docker (optionnel)	5433
Redis Docker (optionnel)	6379

4. Stack technique

Composant	Technologie	Remarque
Frontend	React 18, Vite 7, Tailwind, Framer Motion, Zustand	Port 3002, proxy /api → 5001
Backend	Node.js, Express 5, Sequelize	Port 5001 (standard équipe)
Base de données (dev)	SQLite persistant	backend/data/database.sqlite
Base de données (prod cible)	PostgreSQL 15	docker-compose port 5433
Cache / sessions	Redis 7	Rotation refresh tokens
Temps réel	Socket.IO	Notifications, messages, missions SAV
Offline	Dexie (IndexedDB) + syncService	COD + mode_resilience
Cartes GPS	Leaflet	Suivi transporteur
PWA	vite-plugin-pwa	Service worker en production
Paiements	CinetPay v2 + wallet interne	Simulation par défaut ; staging documenté
IA	Groq API	Assistant achat, scoring crédit, litiges
CI	GitHub Actions	npm audit + test:security + test:phase1

5. Phase 1 — Conception & prototypage ✓

Référence : backend/phase du developpement.md — Livrables : prototype catalogue/commandes/paiements.

5.1 Fondations

- Architecture backend/frontend structurée
- Schéma SQL Phase 1 (UUID, séquestre, split payment par fournisseur)
- Modèles MVP : User, Wallet, Store, Product, Category, Order, OrderItem, Transaction
- Table verifications_otp + service OTP (inscription, paiement, retrait)

5.2 Authentification

- JWT RS256, Google OAuth, 2FA TOTP, rotation refresh token
- RBAC : authMiddleware , grantAccess , authorize
- Routes OTP : POST /auth/otp/request , POST /auth/otp/verify

5.3 Catalogue & commandes

- CRUD produits fournisseur (ownership), gestion boutique, admin catégories
- Panier Zustand, checkout wallet / MM (simulation) / COD
- RBAC : seuls client et admin passent commande
- Séquestre automatique ; stock MM différé jusqu'à confirmation paiement
- Mode hors ligne : COD + flag mode_resilience
- Tests : tests/orders.test.js

6. Phase 2 — Développement pilote ✓

Référence : backend/phase2 du developpement.md — Version bêta testable en environnement pilote.

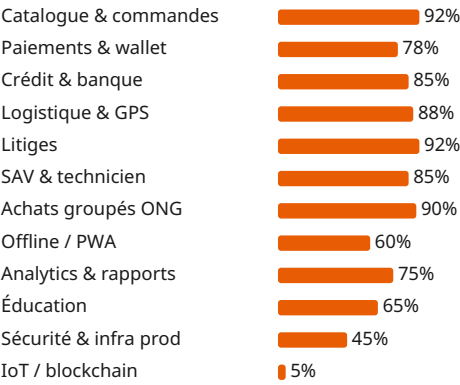
Domaine	Livrables clés	Statut
Logistique	3 tiers livraison, GPS, OTP, livraisons groupées, admin logistique	OK
Litiges	Workflow interactif, LitigeEvenement, médiation IA, remboursement escrow	OK
Crédit & banque	Simulateur, approve/reject, /bank/credits, cron rappels in-app	OK
Achats groupés	Campagnes ONG/B2B, wallet engagement, clôture stock+séquestre	OK
SAV & technicien	Garanties, interventions, missions, notifications socket	OK
Paiements	CinetPay simulation, webhook HMAC, staging doc	Staging prêt
UX pilote	Avis, notifications, ads fournisseur, éducation admin, PWA	OK

7. Phase 3 — Lancement (en cours)

Référence : backend/phase3 du developpement.md — Démarrage après commit 9fece84 .

Item Phase 3	Statut	Détail
docker-compose.yml	Prêt	Postgres 15 + Redis 7 + backend + frontend
scripts/docker-dev.sh	Prêt	Infra db+redis ; détection erreurs Docker
scripts/dev-backend.sh	Prêt	Fallback SQLite automatique
verify-postgres.js	Prêt	Test connexion + migrations
Export PDF/Excel/CSV admin	Prêt	FinancialReports.jsx (xlsx, jsPDF)
SMS rappels crédit	Stub prêt	smsService.js (console/http) + cron
CI test:phase1	Ajouté	40 tests CI (security + phase1)
EQUIPE_DEV.md	Prêt	Guide équipe 2 devs
backend/.env.example	Prêt	Template onboarding
Postgres prod déployé	À faire	Validation migrations prod
CinetPay live	Bloqué	Clés API marchand externes
Provider SMS réel	À faire	Africa's Talking / Twilio / Orange
Plan UAT formalisé	À faire	docs/UAT_PLAN.md
Playwright en CI	Optionnel	20 scénarios documentés

8. État par module fonctionnel



Module	Statut	Commentaire
Marketplace & catalogue	Opérationnel	38 catégories, mega-menu BCA, fiche produit B2B
Commandes & checkout	Opérationnel	3 tiers livraison, RBAC place_orders
Paielements wallet	Opérationnel	Séquestre testé Jest
CinetPay production	Staging prêt	DEPLOYMENT_STAGING.md ; clés marchand requises
Logistique GPS	Opérationnel	OTP livraison, admin /admin/logistics
Litiges	Opérationnel	14 tests Jest ; événements litige
Crédit banque	Opérationnel	Approve → commande + séquestre
Achats groupés	Opérationnel	22/22 tests script
SAV technicien	Opérationnel	Missions, équipements, wallet
Export rapports admin	Opérationnel	Excel, CSV, PDF — FinancialReports
SMS crédit	Stub	Console/http ; provider prod à brancher
Offline PWA	Partiel	COD sync OK ; cache catalogue partiel
IoT / blockchain	Stub	Phase 4

9. État par rôle utilisateur

Rôle	Complétude	Parcours principal
Client	~92 %	Achat, wallet, SAV, litiges, crédit, achats groupés, avis
Fournisseur	~88 %	Produits, commandes, litiges, publicités, rapports
Admin	~90 %	11+ modules, logistique, litiges, éducation, rapports financiers
Transporteur	~88 %	Missions → GPS → OTP → wallet
Banque	~80 %	Dashboard + /bank/credits
Technicien	~85 %	Missions, équipements, wallet

Comptes de test (`npm run seed:accounts`)

Rôle	Email	Mot de passe
Admin	admin@test.com	Admin@123
Client	client@test.com	Client@123
Fournisseur	fournisseur@test.com	Fournisseur@123
Transporteur	transporteur@test.com	Transport@123
Banque	banque@test.com	Banque@123
Technicien	technicien@test.com	Technicien@123

10. Tests et qualité

Suite	Commande	Résultat	Date
Sécurité Jest	npm run test:security	24/24	7 juin 2026
Phase 1 intégration	npm run test:phase1	16/16	7 juin 2026
Achats groupés	test-group-purchases.js	22/22	7 juin 2026
API smoke	api-smoke-test.mjs	7/7	7 juin 2026
Navigateur Playwright	browser-smoke-test.mjs	20/20	Documenté
Total vérifié		69/69	

Couverture tests par domaine

- Auth, refresh token, 2FA, OTP
- Escrow / séquestre (wallet, libération OTP)
- Litiges (workflow, IDOR, remboursement) — 14 tests
- Webhook HMAC paiement CinetPay
- Commandes Phase 1 (RBAC, stock MM, mode_resilience)
- Achats groupés E2E (join, leave, close, wallet) — 22 tests

Scripts npm utiles

Script	Usage
npm run test:security	Suite sécurité complète (24 tests)
npm run test:phase1	Parcours Phase 1 (16 tests)
npm run test:phase2	Security + achats groupés
npm run verify:postgres	Connexion + migrations Postgres
npm run seed:accounts	Comptes test 6 rôles
npm run seed:group-purchases	Données demo achats groupés

11. Sécurité

Mesure	État
JWT RS256 + rotation refresh token (Redis)	Implémenté
Validation DTO (express-validator + Joi env)	Routes critiques
Helmet, CORS, rate limiting	Actif
RBAC backend + frontend (6 rôles)	Aligné
Chiffrement AES-256-GCM	Données sensibles
Séquestre anti-fraude	Testé Jest
IDOR litiges, crédit, commandes, paiements	Corrigé
CI GitHub Actions	audit + test:security + test:phase1
PostgreSQL/Redis production	Docker compose local ; déploiement Phase 3

12. Écarts, risques et recommandations

Bloquants externes

Item	Statut	Action
CinetPay production	Staging documenté	Clés API marchand + HTTPS webhook
Provider SMS	Stub intégré	Contrat Africa's Talking / Twilio
PostgreSQL prod	Scripts prêts	Déploiement + verify:postgres

Risques équipe (2 devs)

- Divergence .env — utiliser .env.example comme référence unique

- **Conflits Git** — `git pull` systématique ; noter les changements dans DEVELOPMENT_LOG
- **Docker optionnel** — ne pas bloquer le dev sur Postgres ; SQLite par défaut
- **Travail Phase 3 local** — committer régulièrement (smsService, scripts, CI)

Recommandations par priorité

Priorité	Action	Responsable suggéré
Immédiat	Committer travail Phase 3 (scripts, SMS, CI, EQUIPE_DEV)	Les 2
Immédiat	Valider dev avec <code>./scripts/dev-backend.sh</code> sur les 2 postes	Les 2
Court terme	Rédiger plan UAT par rôle (docs/UAT_PLAN.md)	Dev B
Court terme	Valider migrations Postgres (verify:postgres)	Dev A (Docker)
Moyen terme	Obtenir clés CinetPay + test staging ngrok	Les 2
Moyen terme	Pilote utilisateurs restreint (Conakry)	Product
Communication	Positionner comme « pilote bêta fonctionnel »	Les 2

13. Parcours de démonstration

1. **Client** : marketplace → panier → checkout wallet → /orders → /tracking
2. **Fournisseur** : valider commande → marquer « prêt »
3. **Transporteur** : accepter mission → GPS → OTP → vérifier wallet
4. **Banque** : /bank/credits → approuver (`npm run seed:pending-credits`)
5. **Litige** : signalement → réponse → résolution admin
6. **SAV** : garanties → maintenance → technicien complète mission
7. **Achats groupés** : /group-purchase → join wallet → clôture
8. **Admin** : /admin/logistics, /admin/disputes, /admin/financial-reports (export PDF)
9. **Hors ligne** : couper réseau → checkout COD → reconnecter → sync

Démarrage local (équipe — recommandé)

```
# Terminal 1 – Backend (SQLite auto)
cd bcaconnect
./scripts/dev-backend.sh

# Terminal 2 – Frontend
cd frontend && npm run dev

# Seeds (première fois ou après migration)
cd backend && npm run seed:accounts
npm run seed:recharge-wallets
```

Démarrage Postgres (optionnel — Phase 3)

```
./scripts/docker-dev.sh          # nécessite accès Docker
cd backend
export DATABASE_URL=postgres://bca_user:bca_password@localhost:5433/bcaconnect
npm run verify:postgres
PORT=5001 npm run dev
```

14. Annexes

14.1 Historique Git récent

Commit	Description
9fece84	Ajout nouvelles fonctionnalités et corrections (Phases 1+2)
aca1112	Documentation présentation Readme.md
0cc55a0	Fusion main + livraisons BCA Connect

Travail Phase 3 local non encore commité : `smsService`, `docker-dev`, `dev-backend`, `verify-postgres`, `EQUIPE_DEV.md`, `CI test:phase1`.

14.2 API — modules montés (/api)

auth, categories, stores, users, products, orders, payments, wallet, delivery, ai, ads, disputes, credits, stats, support, upload, notifications, messages, reviews, sav, education, iot (stub), technician, group-purchases

14.3 Migrations SQL

Fichier	Contenu
20231201_add_technicien_fields.js	Champs rôle technicien
20260602_escrow_and_disputes.js	Séquestre + litiges
20260605_schema_columns.js	Colonnes schéma
20260606_achats_groupes.js	Tables achats groupés
20260607_litige_evenements.js	Événements litige
20260608_order_delivery_type.js	Type livraison commande
20260609_verifications_otp.js	OTP Phase 1

14.4 Documents connexes

- Readme.md — Cahier des charges complet
- EQUIPE_DEV.md — Guide équipe 2 développeurs

- DEVELOPMENT_LOG.md — Journal collaboratif
- STATUS_CONSOLIDE.md — État consolidé
- backend/phase du developpement.md — Phase 1
- backend/phase2 du developpement.md — Phase 2
- backend/phase3 du developpement.md — Phase 3
- DEPLOYMENT_STAGING.md — Staging CinetPay
- backend/DEPLOYMENT_PROD.md — Production
- backend/API_DOCUMENTATION.md — Référence API
- RBAC_MATRIX.md — Matrice permissions

14.5 Verdict final

BCA Connect est une plateforme pilote bêta fonctionnelle et documentée.

Les Phases 1 et 2 du cahier des charges sont clôturées côté développement (commit 9fece84). Le parcours métier principal est validé par **69 tests automatisés**. La Phase 3 (lancement national) est amorcée : scripts infra, SMS stub, exports admin, guide équipe.

Le lancement production nécessite encore : clés CinetPay live, provider SMS, déploiement PostgreSQL/Redis, plan UAT et campagne pilote utilisateurs réels.